

4 - Controllo di versione

Il **controllo di versione** (control version) è una parte della gestione della configurazione del software, che tiene traccia e controlla le modifiche del software che stiamo scrivendo.

Il controllo di versione è utile in diverse situazioni, in particolare:

- Quando più sviluppatori lavorano contemporaneamente allo stesso file (e progetto)
- La manutenzione di vecchie release di software

Comandi universali

Ci sono due comandi universali che sono alla base di ogni software di gestione di controllo:

- `diff` : confronta due file riga per riga e genera un resoconto di tutte le differenze
- `patch` : prende un file originale e un file `.patch` e li unisce per ricreare il file aggiornato.

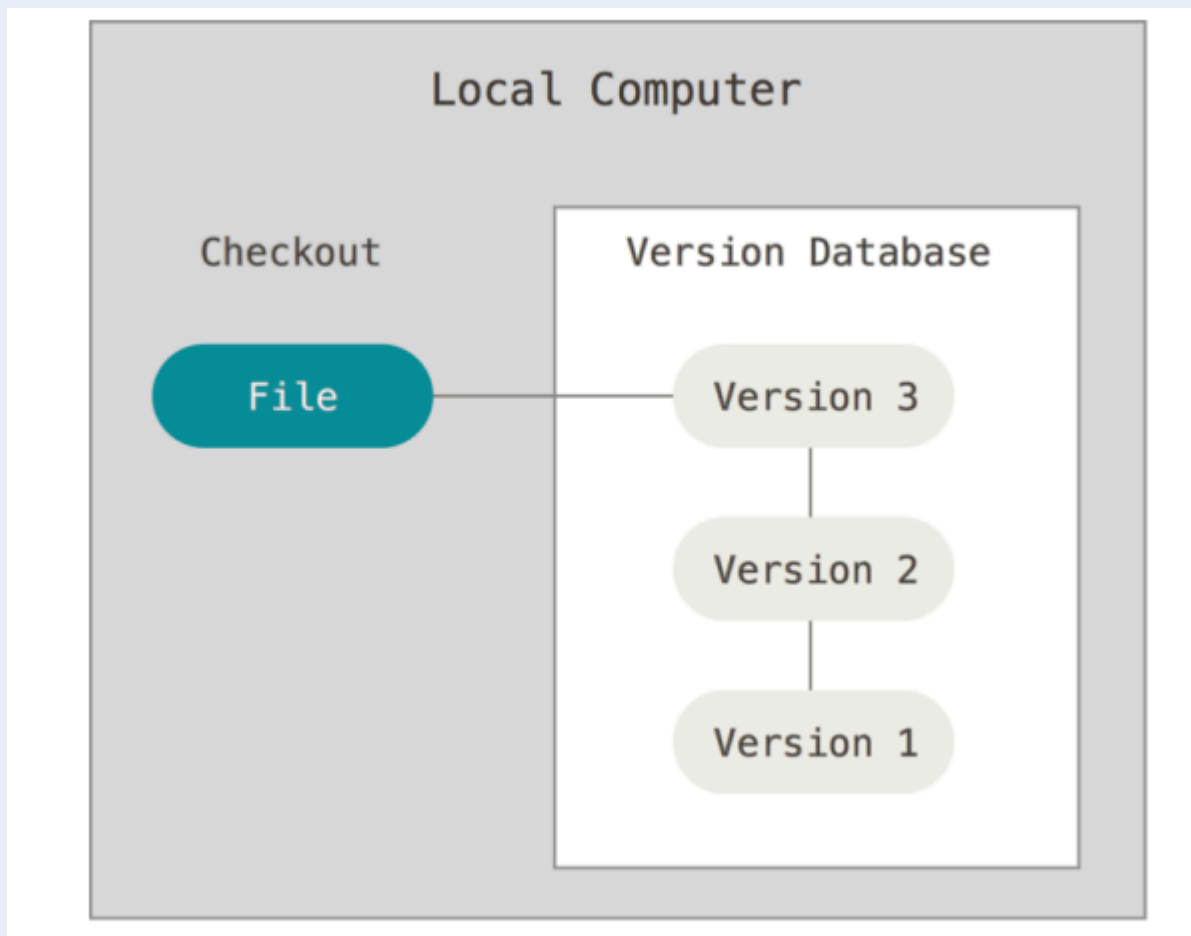
Qualunque sistema quindi che vedremo eredita la logica da questi 2 singoli comandi

Architetture di controllo

Controllo locale

Nel controllo locale, lo sviluppatore ha un database locale (ossia un **repository**) che tiene traccia di tutte le modifiche, non lasciando che siano gestite come file rinominati ma record di un database ad-hoc.

Schema controllo locale

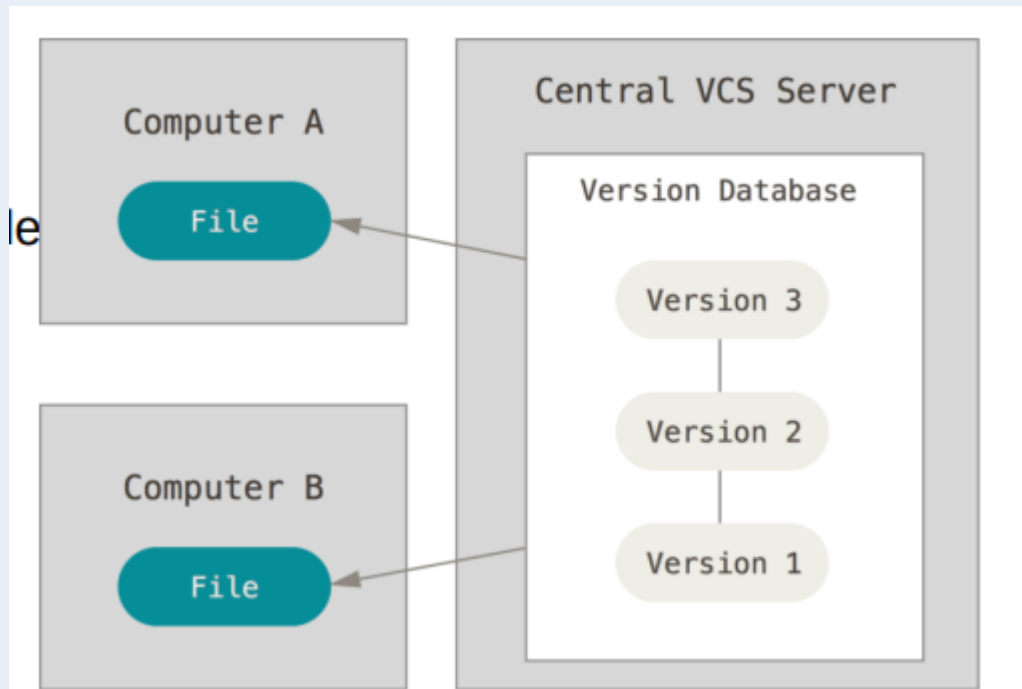


L'unico problema di questa architettura è che non esiste nessun supporto alla collaborazione, quindi supporta il **multi-version** (più versioni) ma non il **multi-user** (più utenti)

Controllo centralizzato di versione

Nel controllo centralizzato di versione, l'architettura si basa su un unico repository condiviso su un server centrale contenente tutti i file versionati

Schema controllo centralizzato di versione



Gli sviluppatori effettuano due operazioni:

- Scaricano una copia locale (**checkout**) dal server al computer locale
- Registrano i file modificati sul repository condiviso nel server (**commit**)

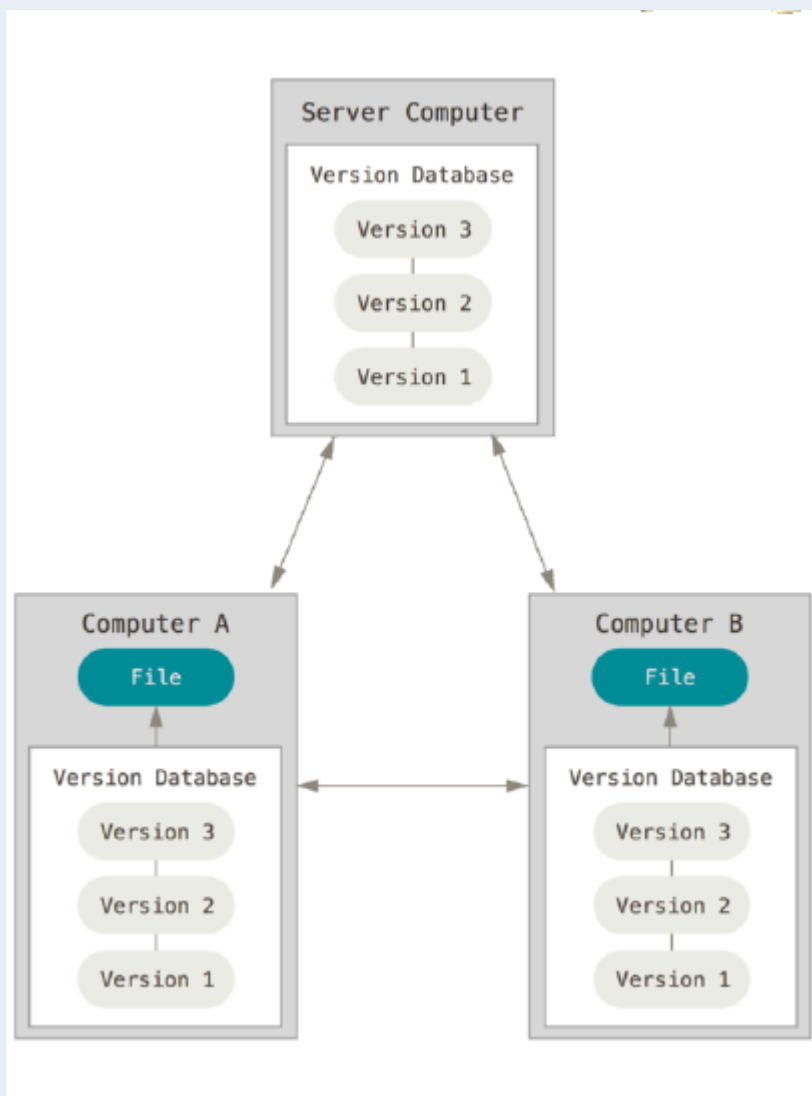
Nel caso ci sia un conflitto in un file (modifiche apportate da due o più persone), chi arriva per ultimo lo risolve (operazione chiamata **merge**).

Questa architettura presenta problematiche nella gestione, in particolare creare un ramo parallelo di sviluppo (chiamato **branch**) spesso significava che il server doveva letteralmente copiare e duplicare l'intera cartella del progetto, per riunirlo con quello principale (**merge**) spesso generava conflitti difficili da aggiustare.

Controllo distribuito di versione

A differenza del modello centralizzato, in questo caso esistono più repository, ognuno dei quali possiede la storia completa di tutte le versioni del software

Schema controllo distribuito di versione



Gli sviluppatori adottano quindi questo workflow (principalmente questo si riferisce a Git, ma è applicabile anche ad altri):

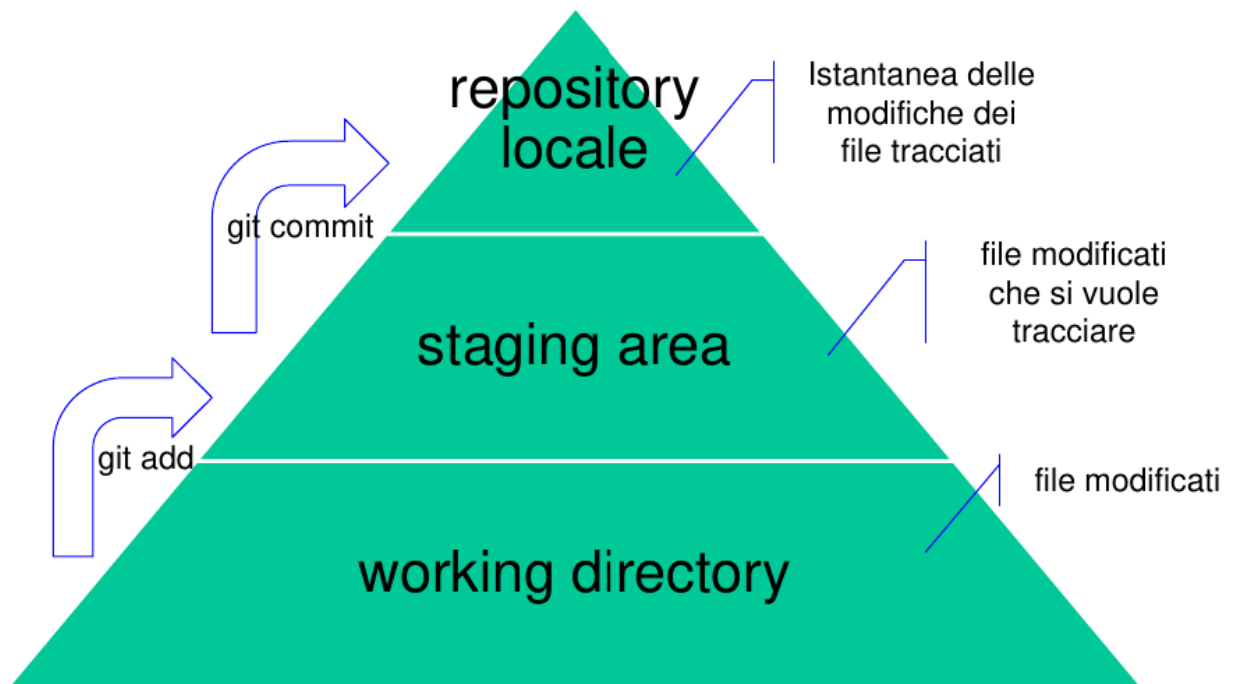
1. Iniziano copiando un intero repository tramite un'operazione di `clone`, oppure ne creano uno nuovo, lavorando poi offline sui file della loro copia locale
2. Successivamente, registrano le modifiche sul proprio repository locale
3. Infine condividono il lavoro inviandolo su un repository remoto tramite il comando di o chiedendo al proprietario di tirare giù le loro modifiche

L'unico vero svantaggio di questo sistema è la curva di apprendimento inizialmente ripida, dovuta ai molteplici flussi di lavoro possibili

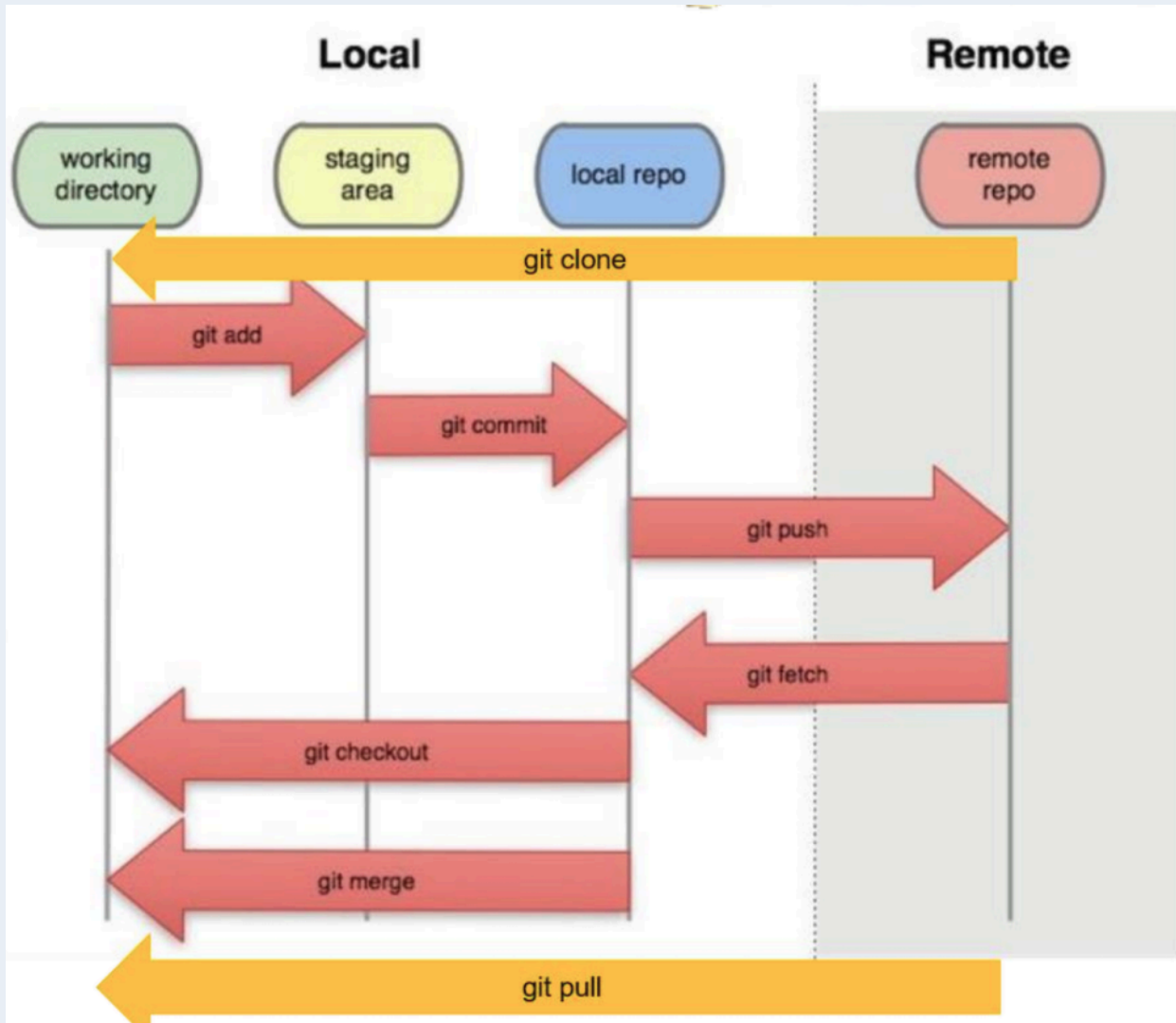
Git

Modifiche nel computer locale

Ogni file, in base al comando, passa in determinate zone come descritte da questo schema:



Prof. Filippa Lanubile



Vari comandi di Git

Categoria	Comando	Descrizione
CONFIGURARE GLI STRUMENTI	<code>\$ git config --global user.name "[name]"</code>	Imposta il nome da associare alle transazioni di commit
	<code>\$ git config --global user.email "[email address]"</code>	Imposta l'email da associare alle transazioni di commit
	<code>\$ git config --global color.ui auto</code>	Attiva la colorazione automatica dell'output della riga di comando
CREARE REPOSITORY	<code>\$ git init [project-name]</code>	Crea un nuovo repository locale con il nome specificato
	<code>\$ git clone [url]</code>	Scarica un progetto e la sua cronologia completa delle versioni
APPORTARE MODIFICHE	<code>\$ git status</code>	Elenca tutti i file nuovi o modificati da sottoporre a commit
	<code>\$ git diff</code>	Mostra le differenze tra file non ancora sottoposti a staging
	<code>\$ git add [file]</code>	Crea un'istantanea del file in preparazione per la creazione della versione
	<code>\$ git diff --staged</code>	Mostra le differenze tra staging e l'ultima versione del file
	<code>\$ git reset [file]</code>	Rimuove il file dallo staging, ma ne conserva i contenuti
	<code>\$ git commit -m "[descriptive message]"</code>	Registra in modo permanente le istantanee dei file nella cronologia delle versioni
RAGGRUPPARE MODIFICHE	<code>\$ git branch</code>	Elenca tutti i rami locali nel repository corrente
	<code>\$ git branch [branch-name]</code>	Crea un nuovo ramo
	<code>\$ git checkout [branch-name]</code>	Passa al ramo specificato e aggiorna la directory di lavoro
	<code>\$ git merge [branch]</code>	Unisce la cronologia del ramo specificato nel ramo corrente
	<code>\$ git branch -d [branch-name]</code>	Elimina il ramo specificato

Categoria	Comando	Descrizione
RIFATTORE NOMI DEI FILE	<code>\$ git rm [file]</code>	Elimina il file dalla directory di lavoro e ne prepara l'eliminazione
	<code>\$ git rm --cached [file]</code>	Rimuove il file dal controllo di versione ma conserva il file locale
	<code>\$ git mv [file-original] [file-renamed]</code>	Cambia il nome del file e lo prepara per il commit
SOPPRIMERE IL TRACCIAMENTO	<code>* .log build/ temp-*</code>	File denominato <code>.gitignore</code> per sopprimere il controllo di versione accidentale
	<code>\$ git ls-files --other --ignored --exclude-standard</code>	Elenca tutti i file ignorati in questo progetto
SALVARE FRAMMENTI	<code>\$ git stash</code>	Memorizza temporaneamente tutti i file tracciati modificati
	<code>\$ git stash pop</code>	Ripristina i file più recenti salvati nello stash
	<code>\$ git stash list</code>	Elenca tutti i set di modifiche memorizzati nello stash
	<code>\$ git stash drop</code>	Elimina il set di modifiche più recenti nello stash
REVISIONE DELLA CRONOLOGIA	<code>\$ git log</code>	Elenca la cronologia delle versioni per il ramo attuale
	<code>\$ git log --follow [file]</code>	Elenca la cronologia delle versioni di un file, compresi i cambi di nome
	<code>\$ git diff [first-branch] ... [second-branch]</code>	Mostra le differenze di contenuto tra due rami
	<code>\$ git show [commit]</code>	Mostra i metadati e le modifiche al contenuto del commit specificato
REDO COMMIT	<code>\$ git reset [commit]</code>	Annulla tutti i commit dopo <code>[commit]</code> , mantenendo le modifiche locali
	<code>\$ git reset --hard [commit]</code>	Elimina tutta la cronologia e torna indietro al commit specificato
SINCRONIZZARE LE MODIFICHE	<code>\$ git fetch [bookmark]</code>	Scarica tutta la cronologia dal segnalibro del repository

Categoria	Comando	Descrizione
	<code>\$ git merge [bookmark]/[branch]</code>	Unisce il ramo del segnalibro nel ramo locale corrente
	<code>\$ git push [alias] [branch]</code>	Carica tutti i commit del ramo locale su GitHub
	<code>\$ git pull</code>	Scarica la cronologia dei segnalibri e incorpora le modifiche

Commenti

I commenti in Git sono molto significativi, poiché aiutano a far capire al team (e a se stessi a distanza di tempo) ciò che si è fatto in una commit.

Ci sono alcune regole generali di **etichette** da seguire, ad esempio:

- Il titolo deve essere separato dalla descrizione con una linea vuota o se il commento è vuoto o piccolo basta inserire solo il titolo.
- Il titolo non deve essere prolisso (massimo 50 caratteri), la descrizione dettagliata è il posto giusto per spiegare meglio i dettagli di un titolo molto lungo
- Il titolo inizia con una lettera maiuscola
- Il titolo è privo di punteggiatura
- Stile imperativo ed in lingua inglese
- La descrizione ha 72 caratteri massimi, oltre quelli è consigliato creare un'altra linea di descrizione
- Bisogna spiegare nella descrizione il **cosa si è fatto** non il **come**.

Branching

Il **branch** ci permette di estrarre dal ramo **madre** una parte in cui ci si può lavorare separatamente, decidendo al termine della linea di sviluppo se inglobarlo nel ramo madre, lasciarlo separato o eliminarlo e creare un ulteriore branch da quest'ultimo.

Questa procedura è il modo migliore per lavorare contemporaneamente a più versioni di un **repository**.

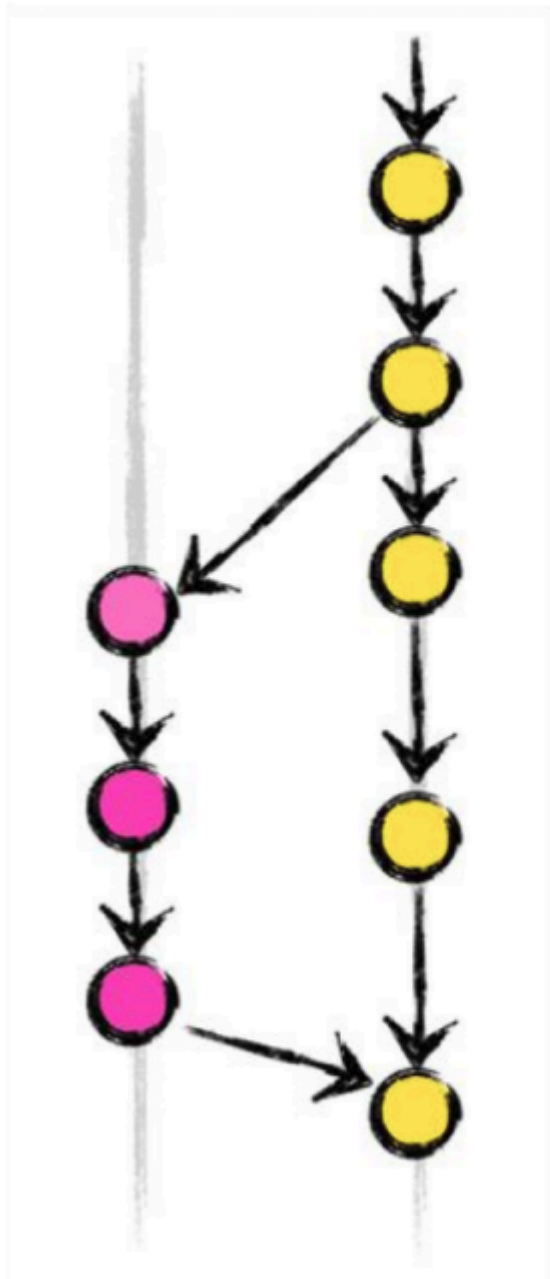
Per default il ramo madre si chiama **master** (oppure **main**) e i branch che vengono riuniti al master verranno rimossi, se quest'ultimi non sono rimossi vuol dire che quel branch serve per tenere traccia comunque di un punto del software .

L'operazione di **unione** del branch lavorato e finito al ramo master in corso (da cui è stato anche separato il branch stesso), si chiama **merge**.

Generalmente il branch viene usato in **locale**, altrimenti si ricadrebbe nello stesso bisogno che si è avuto per creare la separazione tra branch e master.

☰ Rappresentazione grafica del branching

branch X **master**



Comandi per il branching

- `git branch` : mostrare i branch esistenti e visualizzare quello corrente
- `git branch branch-name` : crea un nuovo branch a partire da quello corrente
- `git checkout branch-name` : cambia il branch corrente e aggiorna la working directory
- `git merge branch-name` : fonde il branch specifico con quello madre
- `git merge -d branch-name` : cancella il branch selezionato

Inizializzazione e Analisi

- `git init <nome-cartella>`: Crea e inizializza un nuovo repository Git vuoto nella cartella specificata. Da questo momento, Git inizierà a tracciare le modifiche in quella directory.
- `git status`: Mostra lo stato attuale del repository. Indica in quale branch ci si trova, quali file sono stati modificati, quali sono pronti per il commit (area di staging) e quali non sono ancora tracciati (untracked).
- `git diff`: Mostra nel dettaglio le differenze tra i file nella directory di lavoro e quelli che sono stati preparati nell'area di staging. Utile per vedere esattamente quali righe di codice sono state aggiunte o rimosse prima di fare un `add`.

Salvataggio delle Modifiche (Staging e Commit)

- `git add <nome-file>`: Aggiunge un file (o le sue modifiche) all'**area di staging**. Questo significa che le modifiche sono "in attesa" e pronte per essere impacchettate nel prossimo commit.
- `git commit -m "messaggio descrittivo"`: Prende tutte le modifiche attualmente nell'area di staging e le salva in modo permanente nella cronologia del repository come una nuova "istantanea". Il flag `-m` permette di allegare un messaggio che spiega brevemente cosa è stato fatto.

Visualizzazione della Cronologia

- `git log`: Mostra l'elenco cronologico di tutti i commit effettuati nel branch corrente, includendo l'ID univoco del commit (hash), l'autore, la data e il messaggio.
- `git shortlog`: Mostra un riepilogo più compatto della cronologia, raggruppando i commit in base all'autore.

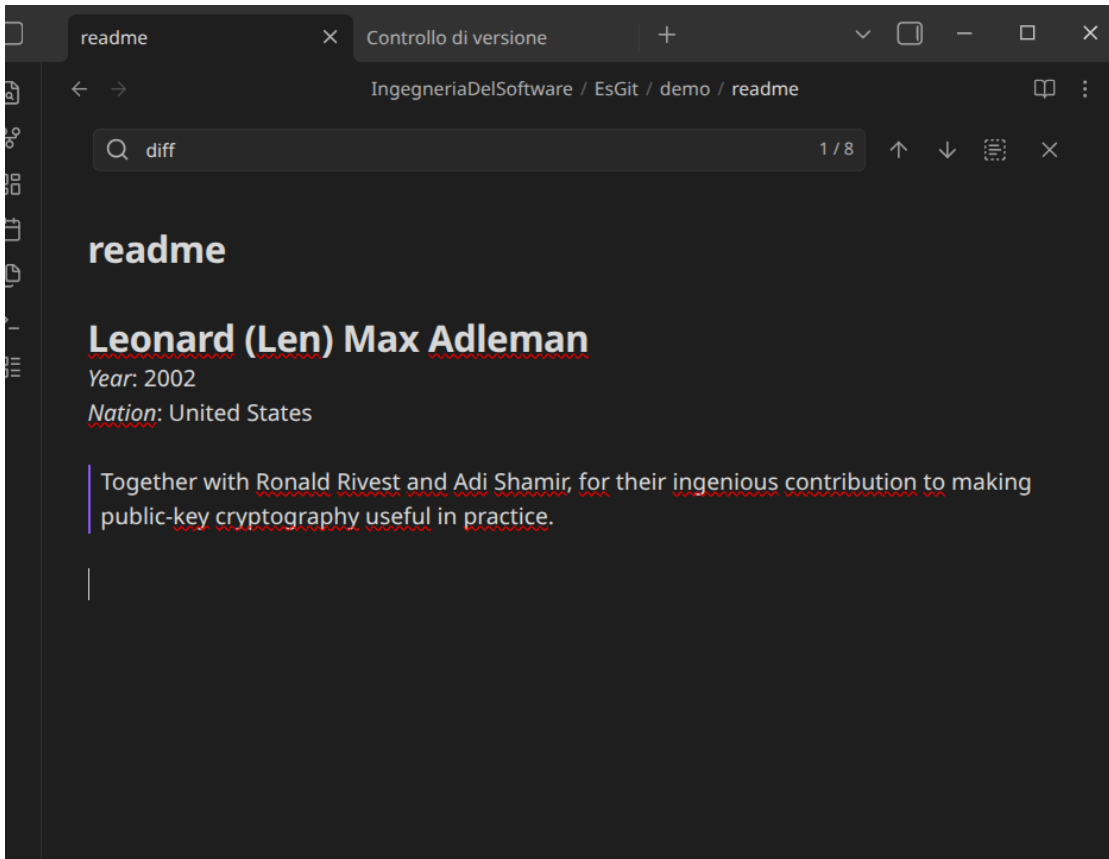
Gestione dei Branch (Branching)

- `git branch`: Se lanciato senza parametri, mostra la lista di tutti i rami (branch) locali presenti nel repository. Un asterisco (`*`) indica il branch su cui ci si trova attualmente (es. `* main`).
- `git branch <nome-branch>`: Crea un nuovo branch con il nome specificato, partendo dal punto esatto della cronologia in cui ci si trova. *Attenzione: questo comando crea solo il branch, ma non ci si sposta automaticamente sopra.*
- `git checkout <nome-branch>`: Cambia il branch attivo, spostando l'ambiente di lavoro (e l'indicatore `HEAD`) sul branch specificato. I file nella directory di lavoro verranno aggiornati per riflettere lo stato di quel branch.
- `git branch -d <nome-branch>`: Elimina un branch locale. L'opzione `-d` (delete) è sicura: Git impedirà di eliminare il branch se contiene modifiche che non sono ancora state unite (merged) da qualche altra parte.

Unione dei Branch (Merging)

- `git merge <nome-branch>`: Unisce le modifiche del branch specificato all'interno del branch in cui ci si trova in quel momento. (Ad esempio, se ci si trova su `main` e si lancia `git merge turingaward`, tutto il lavoro fatto su `turingaward` verrà riversato su `main`).

File MD finale



Github

Una delle difficoltà di Git è sincronizzarsi tra membri, tramite workflow, per capire chi deve attuare le modifiche e quando.

Grazie a **GitHub** possiamo visualizzare il repository remoto condiviso a tutto il team.

Il repository viene **clonato**, copiando l'URL del repository. Quando un repository è clonato, viene messo su una directory specificata dall'utente su cui si potrà poi lavorare localmente e proporre le modifiche su cui mergare.

L'operazione di clone avviene in base a se:

- Il repository è **privato** (bisogna chiedere i diritti e i permessi per accedervi)
- Il repository è **pubblico** (tutto il team è messo allo stesso pari senza gerarchie interne. Anche quest'ultimo però può non essere modificabile in globale poiché solo alcuni membri nell'open-source permettono di modificare un intero progetto e non da tutti)

La **fork** diversamente dal clone, crea una copia dell' URL di quel repository sul proprio GitHub personale, così da modificare totalmente la propria copia.

Issue tracking

Oltre alle operazioni spiegate precedentemente, GitHub favorisce l'**issue tracking**. Esso è un database in cui i record analizzati sono gli issues stessi.

Possono essere classificati come:

- Aperti dal backlog
- Chiusi dal backlog
- Risolti
- Assegnati (in modo che ogni membro del team cerchi di gestire un singolo problema)
- Irrisolti

Gli issues sono anche **numerati** e descritti da **titolo e descrizione**.

In più è possibile creare discussioni asincrone su di essi per parlare del problema con gli utenti.

Sono anche classificati in base al tipo di issues:

- Bug
- Miglioria
- Features
- Security ecc...

Step per lavorare nel team con Github

Analizziamo un workflow, tra quelli usati a livello di default, ovvero il **GitHub Flow**.

Questo workflow permette di poter lavorare in modo **asincrono e parallelo** al progetto ma richiede che alla base ci sia lo stesso repository.

Si eseguono questi 5 passaggi principali:

1. Creare un branch per attuare le modifiche
2. Aggiungere localmente i commit
3. Alla fine del commit aprire la **pull request**
4. Discutere sui cambiamenti attuati
5. **Mergare e deployare**

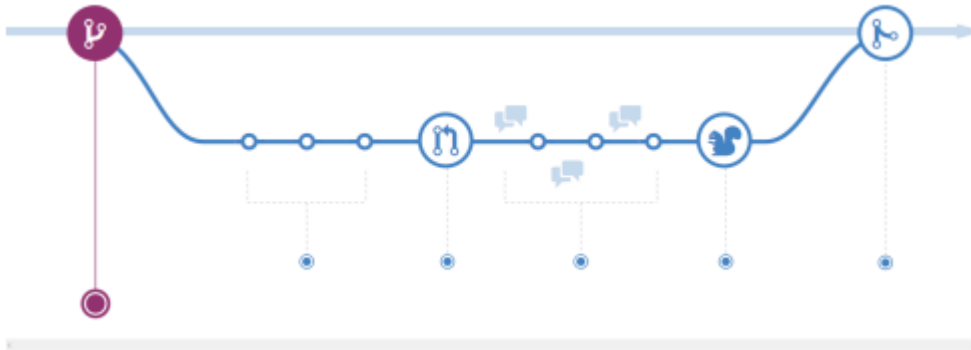
Analizziamo i nuovi termini:

- **Deploy**: Il codice finale presente nel main branch (completo di test e totalmente funzionante) viene reso accessibile, per qualsiasi utilizzo, agli utenti finali.

- **Pull request:** Richiesta formale inviata ai componenti del progetto per discutere le modifiche apportate nel branch secondario.

Fase 1

Bisogna assegnare l'issue su cui si sta lavorando per la modifica come **assigned** e creare il branch sul repository locale nominato con numero dell'issue, titolo e descrizione.



Fase 2

Spostandosi sul branch nuovo e dopo aver effettuato le varie modifiche, è necessario eseguire i commit delle modifiche sul nuovo branch (coi comandi affrontati precedentemente).

Fase 3

La pull request è legata ad un utente del team su cui argomentare l'issue e volendo si può legarla per la chiusura automatica all'issue stesso tramite la dicitura **closes #123**

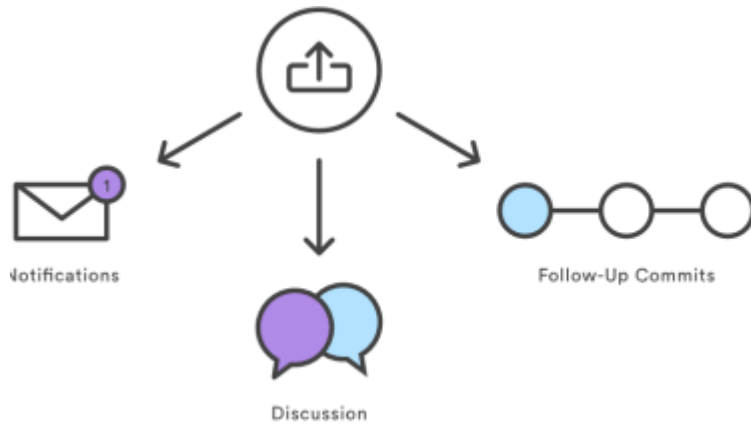
La pull request ha la stessa descrizione dell'issue, se è basata su quest'ultimo, altrimenti deve contenere una descrizione con i test eseguiti, le note e avvertimenti, possibili link utili e se necessario anche immagini. Infine menzionare l'utente (@username) a cui fare la request.



Fase 4

Alla pull request può partecipare chiunque per dire la sua idea. Prima però l'approvazione dell'apertura del dialogo deve essere accettata e discussa con il membro del team

responsabile e poi ci si discute sopra.



In questa discussione è possibile attuare ulteriori modifiche prima del merge.

Fase 5

Alla fine di tutti i check di controllo della qualità, viene cancellato il branch sia via remoto (web) che locale (oppure mantenuto nel caso si voglia avere una storia delle modifiche). Per risolvere i possibili conflitti presenti nel branch principale avviene la ri-esecuzione dei seguenti passi:

- Aggiornare il repository locale
- Modifiche e commit
- Push dei cambiamenti su remoto
- Ritentare il merge

Dopo queste modifiche e merging, il master branch viene deployato in modo **automatica** o manuale.

